

Product Feedback Analyzer

Project Overview

Product Feedback Analyzer is a backend-based web application designed to collect, manage, and analyze customer feedback in a centralized platform. The system enables users to submit feature requests, bug reports, and improvement suggestions, while allowing other users to vote on existing feedback items. These votes help product managers identify the most demanded features and prioritize product development based on actual user needs.

The project demonstrates the practical implementation of MongoDB for document-based data storage, relationships between collections, indexing, aggregation pipelines, and analytics generation. The system is built using Node.js, Express.js, and MongoDB and exposes RESTful APIs that can be tested through Postman and documented using Swagger UI.

Problem Statement

Organizations often receive feedback through multiple channels such as emails, surveys, support tickets, and social media. Managing this feedback manually makes it difficult to identify recurring issues, prioritize feature requests, and track implementation progress.

The Product Feedback Analyzer solves this problem by providing a centralized system where feedback can be submitted, organized, voted upon, tracked, and analyzed through automated dashboards.

Project Objectives

1. Create a centralized platform for collecting customer feedback.
2. Allow users to submit feature requests, bug reports, and improvement suggestions.
3. Enable users to vote on feedback items to indicate demand and priority.
4. Provide product managers with actionable insights using analytics.
5. Track the status of feedback throughout its lifecycle.
6. Demonstrate MongoDB CRUD operations and aggregation pipelines.

7. Implement relationships between collections using MongoDB references.
 8. Generate analytical reports for data-driven product decisions.
-

Key Features

User Management

- Create users
- Retrieve user information
- User role management (User / Manager)

Feedback Management

- Submit feedback
- View all feedback
- View feedback by ID
- Update feedback details
- Delete feedback

Voting System

- Users can vote on feedback items
- Prevent duplicate votes
- Track popularity of requests

Search and Filtering

- Search feedback using keywords
- Filter feedback by category
- Filter feedback by status

Analytics Dashboard

- Total Users
- Total Feedback
- Total Votes

- Open Requests
- Completed Requests

Advanced Analytics

- Most Requested Features
- Category-wise Statistics
- Status-wise Statistics
- Monthly Feedback Trends

API Documentation

- Swagger UI integration
 - Interactive API testing
 - Complete endpoint documentation
-

System Workflow

1. User creates an account.
2. User submits feedback.
3. Feedback is stored in MongoDB.
4. Other users browse available feedback.
5. Users vote on feedback they support.
6. Product managers review submitted feedback.
7. Feedback status is updated.
8. MongoDB aggregation pipelines generate analytical insights.
9. Dashboard displays statistics and trends.
10. Product teams use insights to prioritize future development.

Workflow Summary

User submits feedback → Users vote → Feedback stored in MongoDB → Analytics generated → Product managers prioritize features → Users track progress

Database Design

User Collection

Stores user information.

Fields:

- Name
- Email
- Role
- Created Date

Feedback Collection

Stores feedback submissions.

Fields:

- Title
- Description
- Category
- Status
- Created By
- Created Date

Vote Collection

Stores user votes.

Fields:

- User ID
- Feedback ID
- Created Date

MongoDB Relationships

One-to-Many Relationship

User → Feedback

One user can create multiple feedback entries.

One-to-Many Relationship

Feedback → Votes

One feedback item can receive multiple votes.

Many-to-One Relationship

Vote → User

Many votes can belong to a single user.

Many-to-One Relationship

Vote → Feedback

Many votes can belong to a single feedback item.

Technology Stack

Backend

- Node.js
- Express.js

Database

- MongoDB
- MongoDB Compass
- Mongoose ODM

API Testing

- Postman

API Documentation

- Swagger UI
- Swagger JSDoc

Development Tools

- Visual Studio Code
- Nodemon
- Git
- GitHub

Tools Required

1. Node.js
2. MongoDB Community Server
3. MongoDB Compass
4. Visual Studio Code
5. Postman
6. Git
7. GitHub
8. Swagger UI

REST API Modules

Users API

- Create User
- Get All Users

Feedback API

- Create Feedback
- Get All Feedback
- Get Feedback By ID
- Update Feedback
- Delete Feedback

- Search Feedback

Vote API

- Create Vote
- Get All Votes

Analytics API

- Dashboard Summary
- Most Requested Features
- Category Analytics
- Status Analytics
- Monthly Trends

Expected Outcomes

The Product Feedback Analyzer provides an efficient and scalable solution for collecting and analyzing customer feedback. By combining user submissions, voting mechanisms, MongoDB relationships, and aggregation-based analytics, the system helps organizations make informed product decisions based on customer demand. The project also demonstrates practical implementation of modern backend development concepts, REST APIs, MongoDB operations, and API documentation.

HTTP http://localhost:4000/api/users

POST

http://localhost:4000/api/votes

Params Authorization Headers (8) Body Pre-request Script Tests Settings

none form-data x-www-form-urlencoded raw binary JSON

```
1  {
2    "user": "6a3159fea3d3f1ebedbb0ea7",
3    "feedback": "6a33e85a12cef6a3cebdba0c"
4  }
```

Body Cookies Headers (7) Test Results

Pretty

Raw

Preview

Visualize

JSON

```
1  {
2    "user": "6a3159fea3d3f1ebedbb0ea7",
3    "feedback": "6a33e85a12cef6a3cebdba0c",
4    "_id": "6a33ea6712cef6a3cebdba0d",
5    "createdAt": "2026-06-18T12:53:59.099Z",
6    "updatedAt": "2026-06-18T12:53:59.099Z",
7    "__v": 0
8  }
```


Compass

My Queries

Data Modeling

CONNECTIONS (4)

Search connections

FirstCollection

learning MDB

admin

college

config

ecommerce

firstdb

local

productFeedbackDB

feedbacks

users

votes

secondbrain

test

Project1

Project2

feedbacks

mongodb: learning MDB

users

votes

learning MDB > productFeedbackDB > users

Documents (21)

Aggregations

Schema

Indexes (2)

Validation

Type a query: { field: 'value' } or [Generate query](#)

ADD DATA

UPDATE

DELETE

EXPORT DATA

EXPORT CODE

```
{
  "_id": ObjectId('6a3159fea3d3f1ebedbb0e94'),
  "name": "Tyshawn Ritchie",
  "email": "Harry.Tremblay@yahoo.com",
  "role": "manager",
  "__v": 0,
  "createdAt": 2026-06-16T14:13:18.577+00:00,
  "updatedAt": 2026-06-16T14:13:18.577+00:00
}
```

```
{
  "_id": ObjectId('6a3159fea3d3f1ebedbb0e95'),
  "name": "Clement Hettinger",
  "email": "Gustave63@hotmail.com",
  "role": "user",
  "__v": 0,
  "createdAt": 2026-06-16T14:13:18.579+00:00,
  "updatedAt": 2026-06-16T14:13:18.579+00:00
}
```

```
{
  "_id": ObjectId('6a3159fea3d3f1ebedbb0e96'),
  "name": "Theresa Bogan",
  "email": "Nellie_Medhurst@yahoo.com",
  "role": "user",
  "__v": 0,
  "createdAt": 2026-06-16T14:13:18.579+00:00,
  "updatedAt": 2026-06-16T14:13:18.579+00:00
}
```

```
{
  "_id": ObjectId('6a3159fea3d3f1ebedbb0e97'),
  "name": "Mr. Percy Christiansen",
  "email": "Jared_Hudson@hotmail.com",
  "role": "user",
  "__v": 0,
  "createdAt": 2026-06-16T14:13:18.579+00:00,
  "updatedAt": 2026-06-16T14:13:18.579+00:00
}
```

```
{
  "_id": ObjectId('6a3159fea3d3f1ebedbb0e98'),
  "name": "Reagan Hansen",
  "email": "Daniella72@gmail.com",
  "role": "user",
  "__v": 0,
  "createdAt": 2026-06-16T14:13:18.579+00:00,
  "updatedAt": 2026-06-16T14:13:18.579+00:00
}
```

Compass

My Queries

Data Modeling

CONNECTIONS (4)

Search connections

FirstCollection

learning MDB

admin

college

config

ecommerce

firstdb

local

productFeedbackDB

feedbacks

users

votes

secondbrain

test

Project1

Project2

feedbacks

mongodb: learning MDB

users

votes

+

learning MDB > productFeedbackDB > feedbacks

Documents (51)

Aggregations

Schema

Indexes (1)

Validation

Type a query: { field: 'value' } or [Generate query](#)

ADD DATA

UPDATE

DELETE

EXPORT DATA

EXPORT CODE

```
createdBy: ObjectId('6a3159fea3d3f1ebdbb0ea0')
__v: 0
createdAt: 2026-06-16T14:13:18.596+00:00
updatedAt: 2026-06-16T14:13:18.596+00:00
```

```
_id: ObjectId('6a3159fea3d3f1ebdbb0ed7')
title: "reinvent immersive paradigms"
description: "Textor appello candidus curtus maxime. Cursim tonsor creta cohibeo can..."
category: "Bug Report"
status: "Under Review"
createdBy: ObjectId('6a3159fea3d3f1ebdbb0ea7')
__v: 0
createdAt: 2026-06-16T14:13:18.596+00:00
updatedAt: 2026-06-16T14:13:18.596+00:00
```

```
_id: ObjectId('6a3159fea3d3f1ebdbb0ed8')
title: "integrate robust supply-chains"
description: "Dammatio nam vere absorbeo coerceo ducimus. Testimonium absum solitudo..."
category: "Feature Request"
status: "Open"
createdBy: ObjectId('6a3159fea3d3f1ebdbb0e9b')
__v: 0
createdAt: 2026-06-16T14:13:18.596+00:00
updatedAt: 2026-06-16T14:13:18.596+00:00
```

```
_id: ObjectId('6a3159fea3d3f1ebdbb0ed9')
title: "disintermediate vertical content"
description: "Uter maiores universe. Debeo synagoga ambitus arceo administratio. Tor..."
category: "Bug Report"
status: "In Progress"
createdBy: ObjectId('6a3159fea3d3f1ebdbb0ea7')
__v: 0
createdAt: 2026-06-16T14:13:18.596+00:00
updatedAt: 2026-06-16T14:13:18.596+00:00
```

```
_id: ObjectId('6a3159fea3d3f1ebdbb0eda')
title: "Dark Mode"
description: "Need dark theme support"
category: "Feature Request"
status: "Completed"
createdBy: ObjectId('6a3159fea3d3f1ebdbb0ea7')
```

